

Multiprocessor Operating Systems

Architecture · Threads · Synchronization · Scheduling · Memory · Virtualization

Topics Overview

01 Basic Multiprocessor System Architectures

02 Interconnection Networks & Structures

03 Design Issues

04 Threads & Process

05 Synchronization

06 Processor Scheduling

07 Memory Management

08 Virtualization & Hypervisors

09 Para Virtualization, Memory & I/O Virtualization

01 · Basic Multiprocessor System Architectures

UMA

Uniform Memory Access

- All processors share a single memory
- Equal access time
- Simple, low scalability
- Used in SMP systems

NUMA

Non-Uniform Memory Access

- Each CPU has local memory
- Access time varies by memory location
- Better scalability
- Used in large servers

COMA

Cache-Only Memory Access

- All memory acts as cache
- No fixed home location
- Data migrates to processor that uses it
- Complex hardware

02 · Interconnection Networks & Structures

- Types of Interconnection Networks
 - Bus-Based Networks
 - Crossbar Switch
 - Multistage Networks (Omega, Butterfly)
 - Hypercube Networks
- Key Structures
 - Shared Memory Structure
 - Message Passing Structure
 - Hybrid Structures

Bus

Simple, low cost
Limited bandwidth

Crossbar

High bandwidth
Expensive, $O(N^2)$

Omega/ Butterfly

$\log N$ stages
Good scalability

Hypercube

N -dim structure
Fault tolerant

03 · Design Issues

Cache Coherence

Ensuring all CPUs see consistent data values across caches

Memory Consistency

Defining the order of memory operations visible to all processors

Synchronization

Coordinating access to shared resources using locks & barriers

Scalability

Maintaining performance as number of processors increases

Load Balancing

Distributing work evenly across all available processors

Fault Tolerance

Handling processor or memory failures without crashing the system

04 · Threads & Process

PROCESS

- Independent unit of execution
- Has its own address space
- Heavyweight — high context switch cost
- Isolation via virtual memory
- Inter-process communication needed for data sharing

VS

THREAD

- Lightweight unit of execution
- Shares process address space
- Low context switch overhead
- Faster inter-thread communication
- Types: User, Kernel, Hybrid

05 · Process Synchronization

Mutex Locks

Provides mutual exclusion; only one thread enters critical section at a time.

Semaphores

Counting semaphore controls access to a shared resource with a counter.

Monitors

High-level abstraction with condition variables for structured synchronization.

Barriers

All threads must reach the barrier before any can proceed further.

Spinlocks

Busy-wait lock suitable for very short critical sections on multiprocessors.

Read-Write Locks

Multiple readers OR a single writer allowed; optimizes read-heavy workloads.

06 · Processor Scheduling

01

Gang Scheduling

Schedules all threads of a process simultaneously on multiple CPUs for better performance.

02

Space Sharing

Partitions CPUs among processes; each process gets a dedicated subset of processors.

03

Dynamic Scheduling

Number of CPUs assigned to a process changes based on system load and availability.

04

Work Stealing

Idle processors steal tasks from busy processors' queues to improve load balancing.

07 · Memory Management

- **Shared Memory Multiprocessors**
 - All processors access a global address space
 - OS manages coherency protocols (MESI, MOESI)
- **Distributed Memory**
 - Each node has private memory
 - Communication via message passing (MPI)
- **Virtual Memory in Multiprocessors**
 - Each process has its own virtual address space
 - TLB shutdowns on context switch
- **NUMA-Aware Memory Management**
 - OS places memory close to the processor using it
 - First-touch policy for page allocation

Cache Hierarchy

L1 / L2 / L3

Page Table

Per Process

Allocation

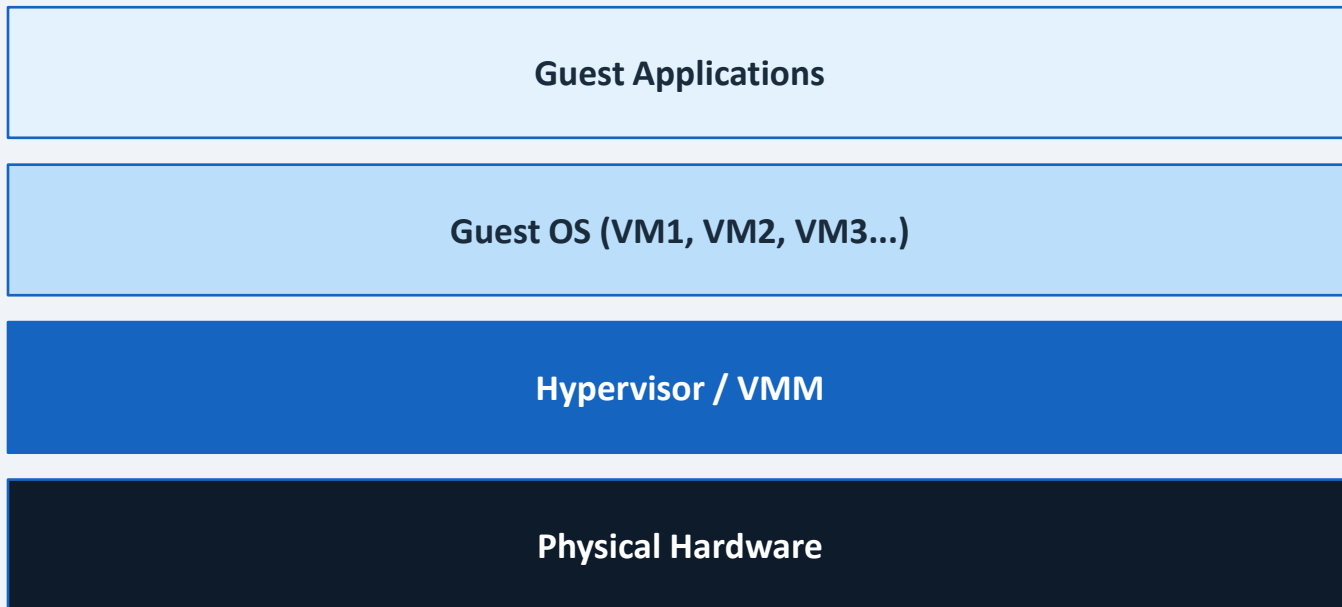
First-Touch

Protocol

MESI / MOESI

08 · Virtualization

Virtualization creates an abstraction layer between hardware and software, allowing multiple OS instances to run on a single physical machine.



Benefits: Resource isolation · Consolidation · Snapshots · Live Migration · Security Sandboxing

08 · Types of Hypervisors

Type 1 – Bare Metal

- Runs directly on hardware
- No host OS required
- High performance & security
- Examples:
 - VMware ESXi
 - Microsoft Hyper-V
 - Xen Hypervisor
 - KVM (Linux kernel)

Type 2 – Hosted

- Runs on top of a host OS
- Easier to install & use
- Lower performance overhead
- Examples:
 - VMware Workstation
 - Oracle VirtualBox
 - QEMU (user mode)
 - Parallels Desktop

08 · Para Virtualization

- What is Para-Virtualization?
 - Guest OS is modified to communicate directly with the hypervisor
 - Uses hypercalls instead of emulated hardware instructions
- Advantages
 - Lower overhead than full virtualization
 - Better I/O and memory performance
 - More efficient use of hardware resources
- Disadvantages
 - Requires modification of the guest OS kernel
 - Cannot run unmodified proprietary OS (e.g., Windows)
- Key Example
 - Xen Hypervisor (uses para-virtualized drivers)

Comparison		
Feature	Full Virt.	Para Virt.
Guest Mod.	No	Yes
Performance	Lower	Higher
Complexity	Less	More
Portability	High	Limited

09 · Memory Virtualization & I/O Virtualization

Memory Virtualization

- Guest OS manages virtual addresses
- Hypervisor translates to physical addresses
- Techniques:
 - Shadow Page Tables – hypervisor maintains separate page tables
 - Hardware-Assisted (EPT/NPT) – AMD-V and Intel VT-x support
 - 2-level address translation
 - Balloon Driver – reclaims unused guest memory

I/O Virtualization

- Allows VMs to share physical I/O devices
- Approaches:
 - Emulation – hypervisor emulates a hardware device in software
 - Para-virtualized I/O – guest uses special drivers (virtio)
 - Direct Assignment (SR-IOV) – hardware device directly assigned to a VM
- Devices: Network, Disk, GPU, USB

Summary

Architectures

UMA, NUMA, COMA differ in memory access uniformity

Design Issues

Coherence, Consistency, Scalability, Load Balancing

Synchronization

Mutex, Semaphores, Monitors, Barriers, Spinlocks

Memory Mgmt

Shared, NUMA-aware, Virtual Memory, TLB Shutdown

Networks

Interconnects: Bus, Crossbar, Omega, Hypercube

Threads

Lightweight vs. Process — shared vs. isolated memory

Scheduling

Gang, Space Sharing, Dynamic, Work Stealing

Virtualization

Type 1 / Type 2 Hypervisors, Para-virt, Memory & I/O Virt